

Chapter 34 — Dynamic Programming Foundations

0. What You Will Learn in This Chapter

Dynamic Programming, or DP, is one of the most important and most difficult interview patterns.

DP is used when a problem can be broken into smaller overlapping subproblems, and the answer to a bigger problem depends on answers to smaller problems.

By the end of this chapter, you should understand:

- What dynamic programming is
- Why DP exists
- When to use DP
- When not to use DP
- Overlapping subproblems
- Optimal substructure
- State definition
- Recurrence relation
- Base cases
- Memoization
- Tabulation
- Space optimization
- How to convert recursion to DP
- How to identify DP in interviews
- How to avoid common DP mistakes
- How to build DP solutions from brute force

This chapter is a foundation chapter. It teaches the thinking process.

Later chapters will cover:

- 1D DP
- 2D DP
- String DP
- Knapsack DP
- Interval DP
- Bitmask DP
- Tree DP
- Digit DP
- DP optimization patterns

Core problems covered in this chapter:

1. Fibonacci Number
2. Climbing Stairs
3. Min Cost Climbing Stairs
4. House Robber
5. Unique Paths
6. Coin Change
7. Longest Increasing Subsequence
8. Word Break
9. Decode Ways
10. Triangle
11. Maximum Subarray, DP view

- 12. [Best Time to Buy and Sell Stock with Cooldown, preview](#)
 - 13. [Edit Distance, preview](#)
 - 14. [Partition Equal Subset Sum, preview](#)
 - 15. [Burst Balloons, preview](#)
-

1. What Is Dynamic Programming?

Dynamic Programming is a technique for solving problems by storing answers to smaller subproblems and reusing them.

Instead of recomputing the same thing again and again, DP saves the result.

Example:

Fibonacci:

$$\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$$

Naive recursion recomputes many values.

For $\text{fib}(5)$:

$$\begin{aligned}\text{fib}(5) \\ &= \text{fib}(4) + \text{fib}(3)\end{aligned}$$

$$\begin{aligned}\text{fib}(4) \\ &= \text{fib}(3) + \text{fib}(2)\end{aligned}$$

$\text{fib}(3)$ is computed multiple times.

DP stores it once.

2. Why Dynamic Programming Exists

Many brute-force recursive solutions explore all choices.

But often the same state appears repeatedly.

Example:

Climbing stairs.

To reach step n , you can come from:

$n - 1$

$n - 2$

So:

$$\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$$

Naive recursion branches exponentially.

DP stores each $\text{ways}(i)$ once.

This turns exponential time into polynomial time.

3. Core DP Requirements

A problem is a strong DP candidate if it has both:

3.1 Overlapping Subproblems

The same smaller problems repeat.

Example:

fib(5)
calls fib(4) and fib(3)
fib(4) also calls fib(3)

So fib(3) overlaps.

3.2 Optimal Substructure

The answer to the full problem can be built from answers to smaller problems.

Example:

$\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$

Example:

$\text{minCost}(i) = \text{cost}[i] + \min(\text{minCost}(i - 1), \text{minCost}(i - 2))$

4. DP vs Recursion

Recursion is a way to define a problem in terms of smaller versions.

DP is recursion plus memory, or iterative table-building.

Reuses results?	no	yes
May be exponential?	yes	usually optimized
Stores states?	no	yes
Style	top-down	top-down or bottom-up
Example	naive Fibonacci	memoized Fibonacci

5. DP vs Greedy

Greedy makes one local choice.

DP tries multiple possible choices through states.

Choices considered	one best local	many
Requires local proof	yes	recurrence proof
Can revise decisions	no	yes
Example	Jump Game	Coin Change
Common output	min/max/count/possible	min/max/count/possible

Coin Change is DP because picking the largest coin first can fail.

6. DP vs Backtracking

Backtracking generates all possibilities.

DP avoids recomputing repeated states.

Example:

Combination Sum generates actual combinations → backtracking.

Coin Change minimum number of coins → DP.

Goal	enumerate valid solutions	optimize/count/check
Repeated states	often possible	cached
Output	list of solutions	value / boolean / count
Example	Subsets	Coin Change

7. When to Use DP

Use DP when the problem asks for:

7.1 Minimum / Maximum

Examples:

- minimum cost
- maximum profit
- longest subsequence
- minimum operations

7.2 Count Ways

Examples:

- number of ways to climb stairs
- number of paths
- number of decodings

7.3 True / False Possibility

Examples:

- can partition
- can break word
- can reach target

7.4 Longest / Shortest Sequence

Examples:

- longest increasing subsequence
- longest common subsequence
- shortest edit distance

7.5 Repeated Choices

Examples:

- choose coin
- choose item

- choose split point
 - choose previous state
-

8. When NOT to Use DP

Do not force DP when:

8.1 Greedy Has a Safe Proof

Example:

Jump Game can be greedy.

8.2 Problem Requires Generating All Solutions

Use backtracking.

8.3 No Overlapping Subproblems

Divide and conquer may be enough.

Example:

Merge Sort.

8.4 Simple One-Pass State Is Enough

Example:

Best Time to Buy and Sell Stock I.

8.5 Constraints Are Too Large for State Space

If DP states are too many, need optimization or another pattern.

9. The DP Thinking Framework

For every DP problem, ask these 6 questions:

Question 1 — What is the state?

What does $dp[i]$ or $dp[i][j]$ mean?

Question 2 — What is the transition?

How do I compute this state from previous states?

Question 3 — What are the base cases?

What values are known initially?

Question 4 — What is the answer?

Where is final answer stored?

Question 5 — What is the order of computation?

Which states must be computed before this state?

Question 6 — Can space be optimized?

Do I need the full table or only recent states?

10. What Is a DP State?

A state is a subproblem.

Example:

Climbing Stairs:

$dp[i]$ = number of ways to reach step i
--

House Robber:

$dp[i]$ = maximum money robbable from houses 0..i

Unique Paths:

$dp[r][c]$ = number of ways to reach cell (r, c)

Coin Change:

$dp[amount]$ = minimum coins needed to make amount

The hardest part of DP is often defining the state.

11. What Is a Transition?

A transition explains how to compute one state from smaller states.

Example:

Climbing Stairs:

$dp[i] = dp[i - 1] + dp[i - 2]$

House Robber:

$dp[i] = \max(dp[i - 1], dp[i - 2] + \text{nums}[i])$

Coin Change:

$dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$

Unique Paths:

$dp[r][c] = dp[r - 1][c] + dp[r][c - 1]$

12. What Are Base Cases?

Base cases are known starting states.

Examples:

Climbing Stairs:

$dp[1] = 1$

$dp[2] = 2$

Fibonacci:

$dp[0] = 0$

$dp[1] = 1$

Coin Change:

$dp[0] = 0$

Unique Paths:

first row = 1

first column = 1

Without correct base cases, DP fails.

13. Top-Down DP: Memoization

Top-down DP starts with recursion and adds cache.

Example:

```
int solve(state) {  
    if (base case) {  
        return base value;  
    }  
  
    if (memo contains state) {  
        return memo[state];  
    }  
  
    memo[state] = answer from smaller states;  
  
    return memo[state];  
}
```

This is often easier to derive.

14. Bottom-Up DP: Tabulation

Bottom-up DP fills a table from smaller states to larger states.

Example:

```
dp[0] = 0;  
dp[1] = 1;  
  
for (int i = 2; i <= n; i++) {  
    dp[i] = dp[i - 1] + dp[i - 2];  
}
```

This often uses less recursion stack and can be easier to optimize.

15. Memoization vs Tabulation

Direction	top-down	bottom-up
Starts from	target answer	base cases
Uses recursion	yes	no
Computes unused states?	sometimes no	often yes
Stack overflow risk	yes	no

Feature	Memoization	Tabulation
Easier to write first?	often yes	sometimes harder
Easier to optimize space?	usually no	yes

16. DP State Dimension

16.1 1D DP

State uses one variable.

Examples:

```
dp[i]
dp[amount]
dp[index]
```

Problems:

- Climbing Stairs
- House Robber
- Coin Change
- LIS

16.2 2D DP

State uses two variables.

Examples:

```
dp[i][j]
dp[row][col]
dp[index][target]
```

Problems:

- Unique Paths
- LCS
- Edit Distance
- Knapsack

16.3 3D or Higher DP

State uses more variables.

Examples:

```
dp[index][transactions][holding]
dp[row][col][remaining]
dp[mask][last]
```

Used in:

- stock DP
- grid with constraints
- bitmask DP

Higher-dimensional DP requires careful constraints.

17. How to Derive DP From Brute Force

Steps:

16. Write recursive brute force.
17. Identify repeated state parameters.
18. Cache result for those parameters.
19. Convert to table if needed.
20. Optimize space if possible.

Example:

Coin Change recursion:

```
solve(amount) = min coins to make amount
```

Choices:

```
try every coin
```

Recurrence:

```
solve(amount) = 1 + min(solve(amount - coin))
```

Memoize amount.

18. Full Problem Walkthrough 1 — LeetCode 509: Fibonacci Number

18.1 Problem Summary

Compute nth Fibonacci number:

```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n - 1) + fib(n - 2)
```

18.2 How to Identify DP

Naive recursion repeats states.

fib(3) appears many times.

Use memoization or tabulation.

18.3 Brute Force Recursion

```
class Solution {
public:
    int fib(int n) {
        if (n <= 1) {
            return n;
        }
    }
}
```

```
        return fib(n - 1) + fib(n - 2);
    }
};
```

Time:

$O(2^n)$

18.4 Memoization

```
class Solution {
public:
    int solve(int n, vector<int>& memo) {
        if (n <= 1) {
            return n;
        }

        if (memo[n] != -1) {
            return memo[n];
        }

        memo[n] = solve(n - 1, memo) + solve(n - 2, memo);

        return memo[n];
    }

    int fib(int n) {
        vector<int> memo(n + 1, -1);
        return solve(n, memo);
    }
};
```

Time:

$O(n)$

Space:

$O(n)$

18.5 Tabulation

```
class Solution {
public:
    int fib(int n) {
        if (n <= 1) {
            return n;
        }
    }
```

```

    }

    vector<int> dp(n + 1, 0);

    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}
};

```

18.6 Space Optimized

```

class Solution {
public:
    int fib(int n) {
        if (n <= 1) {
            return n;
        }

        int prev2 = 0;
        int prev1 = 1;

        for (int i = 2; i <= n; i++) {
            int curr = prev1 + prev2;
            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
};

```

18.7 Core Takeaway

Fibonacci teaches the basic DP pipeline:

recursion -> memoization -> tabulation -> space optimization

19. Full Problem Walkthrough 2 —

LeetCode 70: Climbing Stairs

19.1 Problem Summary

You can climb 1 or 2 steps.

Return number of distinct ways to reach step n .

19.2 State

$dp[i]$ = number of ways to reach step i

19.3 Transition

To reach step i , previous step can be:

$i - 1$

$i - 2$

So:

$dp[i] = dp[i - 1] + dp[i - 2]$

19.4 Base Cases

$dp[1] = 1$

$dp[2] = 2$

19.5 C++ Code

```
class Solution {
public:
    int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }

        int prev2 = 1;
        int prev1 = 2;

        for (int i = 3; i <= n; i++) {
            int curr = prev1 + prev2;
            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
};
```

19.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

20. Full Problem Walkthrough 3 — LeetCode 746: Min Cost Climbing Stairs

20.1 Problem Summary

You are given cost array.

You can climb 1 or 2 steps.

You can start at step 0 or 1.

Return minimum cost to reach the top.

20.2 State

$dp[i]$ = minimum cost to reach step i

The top is step n .

20.3 Transition

To reach step i , you come from:

$i - 1$

$i - 2$

If you come from step $i - 1$, you pay $cost[i - 1]$.

If you come from step $i - 2$, you pay $cost[i - 2]$.

$$dp[i] = \min(dp[i - 1] + cost[i - 1], \\ dp[i - 2] + cost[i - 2])$$

20.4 Base Cases

$dp[0] = 0$

$dp[1] = 0$

20.5 C++ Code

```
class Solution {
public:
    int minCostClimbingStairs(vector<int>& cost) {
        int n = cost.size();

        int prev2 = 0;
```

```
int prev1 = 0;

for (int i = 2; i <= n; i++) {
    int curr = min(prev1 + cost[i - 1],
                  prev2 + cost[i - 2]);

    prev2 = prev1;
    prev1 = curr;
}

return prev1;
};
```

20.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

21. Full Problem Walkthrough 4 — LeetCode 198: House Robber

21.1 Problem Summary

You are given houses with money.

Cannot rob adjacent houses.

Return maximum money.

21.2 State

$dp[i]$ = maximum money from houses 0..i

21.3 Choices

For house i:

21. Skip it:

$dp[i - 1]$

22. Rob it:

$dp[i - 2] + nums[i]$

21.4 Transition

$dp[i] = \max(dp[i - 1], dp[i - 2] + nums[i])$

21.5 C++ Code

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int prev2 = 0;
        int prev1 = 0;

        for (int money : nums) {
            int curr = max(prev1, prev2 + money);

            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
};
```

21.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

21.7 DP Thinking Lesson

House Robber is a classic example of:

at every index, choose or skip

22. Full Problem Walkthrough 5 — LeetCode 62: Unique Paths

22.1 Problem Summary

Robot starts at top-left of grid.

Can move only:

right or down

Return number of unique paths to bottom-right.

22.2 State

$dp[r][c]$ = number of ways to reach cell (r, c)

22.3 Transition

To reach (r, c) , robot can come from:

(r - 1, c)
(r, c - 1)

So:

$dp[r][c] = dp[r - 1][c] + dp[r][c - 1]$

22.4 Base Cases

First row:

1 way

First column:

1 way

22.5 C++ Code

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, 1));

        for (int r = 1; r < m; r++) {
            for (int c = 1; c < n; c++) {
                dp[r][c] = dp[r - 1][c] + dp[r][c - 1];
            }
        }

        return dp[m - 1][n - 1];
    }
};
```

22.6 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

Space can be optimized to $O(n)$.

23. Full Problem Walkthrough 6 — LeetCode 322: Coin Change

23.1 Problem Summary

Given coins and amount.

Return minimum number of coins needed to make amount.

If impossible, return -1.

23.2 How to Identify DP

At every amount, we can choose any coin.

Need minimum count.

Greedy fails for some coin systems.

23.3 State

$dp[a]$ = minimum coins needed to make amount a

23.4 Base Case

$dp[0] = 0$

23.5 Transition

For every amount a and coin:

$dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$

if $a \geq \text{coin}$.

23.6 C++ Code

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        const int INF = 1e9;

        vector<int> dp(amount + 1, INF);
        dp[0] = 0;

        for (int a = 1; a <= amount; a++) {
            for (int coin : coins) {
                if (a >= coin && dp[a - coin] != INF) {
                    dp[a] = min(dp[a], dp[a - coin] + 1);
                }
            }
        }

        return dp[amount] == INF ? -1 : dp[amount];
    }
};
```

23.7 Complexity

Time:

$O(\text{amount} * \text{numberOfCoins})$

Space:

$O(\text{amount})$

24. Full Problem Walkthrough 7 — LeetCode 300: Longest Increasing Subsequence

24.1 Problem Summary

Given nums, return length of longest strictly increasing subsequence.

24.2 DP State

$dp[i]$ = length of longest increasing subsequence ending at i

24.3 Transition

For every $j < i$:

If:

$nums[j] < nums[i]$

then:

$dp[i] = \max(dp[i], dp[j] + 1)$

24.4 C++ Code $O(n^2)$

```
class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        int n = nums.size();

        vector<int> dp(n, 1);
        int answer = 1;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    dp[i] = max(dp[i], dp[j] + 1);
                }
            }

            answer = max(answer, dp[i]);
        }

        return answer;
    }
};
```

24.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

24.6 Note

There is also an $O(n \log n)$ binary-search solution.

That belongs to advanced DP / binary search overlap.

25. Full Problem Walkthrough 8 — LeetCode 139: Word Break

25.1 Problem Summary

Given string s and dictionary, return true if s can be segmented into dictionary words.

25.2 State

$dp[i]$ = true if $s[0..i-1]$ can be segmented

25.3 Base Case

$dp[0]$ = true

Empty string can be segmented.

25.4 Transition

For every $j < i$:

If:

$dp[j]$ == true
and $s[j..i-1]$ is in dictionary

then:

$dp[i]$ = true

25.5 C++ Code

```
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> dict(wordDict.begin(), wordDict.end());

        int n = s.size();
        vector<bool> dp(n + 1, false);

        dp[0] = true;
```

```

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.count(s.substr(j, i - j))) {
                dp[i] = true;
                break;
            }
        }
    }

    return dp[n];
}
};

```

25.6 Complexity

Time:

$O(n^3)$

because substring creation can cost $O(n)$.
 Can be optimized using word lengths or trie.

26. Full Problem Walkthrough 9 — LeetCode 91: Decode Ways

26.1 Problem Summary

String contains digits.

Map:

1 -> A
 2 -> B
 ...
 26 -> Z

Return number of ways to decode.

26.2 State

$dp[i]$ = number of ways to decode first i characters

26.3 Transition

Single digit:

If $s[i - 1] \neq '0'$:

$dp[i] += dp[i - 1]$

Two digits:

If $10 \leq \text{value} \leq 26$:

$dp[i] += dp[i - 2]$

26.4 C++ Code

```
class Solution {
public:
    int numDecodings(string s) {
        int n = s.size();

        vector<int> dp(n + 1, 0);
        dp[0] = 1;

        for (int i = 1; i <= n; i++) {
            if (s[i - 1] != '0') {
                dp[i] += dp[i - 1];
            }

            if (i >= 2) {
                int value = stoi(s.substr(i - 2, 2));

                if (value >= 10 && value <= 26) {
                    dp[i] += dp[i - 2];
                }
            }
        }

        return dp[n];
    }
};
```

26.5 Edge Cases

"0" -> 0
"10" -> 1
"06" -> 0
"226" -> 3

26.6 Complexity

Time:

$O(n)$

Space:

$O(n)$

27. Full Problem Walkthrough 10 — LeetCode 120: Triangle

27.1 Problem Summary

Given triangle array, find minimum path sum from top to bottom.
At each step, move to adjacent number in next row.

27.2 State

$dp[r][c]$ = minimum path sum from cell (r,c) to bottom

27.3 Bottom-Up Transition

$dp[r][c] = \text{triangle}[r][c] + \min(dp[r+1][c], dp[r+1][c+1])$

27.4 C++ Code

```
class Solution {
public:
    int minimumTotal(vector<vector<int>>& triangle) {
        vector<int> dp = triangle.back();

        for (int r = triangle.size() - 2; r >= 0; r--) {
            for (int c = 0; c < triangle[r].size(); c++) {
                dp[c] = triangle[r][c] + min(dp[c], dp[c + 1]);
            }
        }

        return dp[0];
    }
};
```

27.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

28. Full Problem Walkthrough 11 — LeetCode 53: Maximum Subarray, DP View

28.1 Problem Summary

Find maximum sum of a contiguous subarray.

28.2 State

$dp[i]$ = maximum subarray sum ending at index i

28.3 Transition

At index i , either:

- start new subarray at i
- extend previous subarray

$dp[i] = \max(nums[i], dp[i - 1] + nums[i])$

28.4 C++ Code

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int current = nums[0];
        int best = nums[0];

        for (int i = 1; i < nums.size(); i++) {
            current = max(nums[i], current + nums[i]);
            best = max(best, current);
        }

        return best;
    }
};
```

28.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

29. Full Problem Walkthrough 12 — Best Time to Buy and Sell Stock With Cooldown, Preview

29.1 Problem Summary

You can buy and sell stocks.

After selling, you must cooldown one day before buying again.

29.2 Why This Is DP

At each day, your state depends on whether you are:

- holding stock
- not holding stock
- in cooldown

29.3 State

$hold[i]$ = max profit on day i while holding stock

$sold[i]$ = max profit on day i after selling

$rest[i]$ = max profit on day i doing nothing / cooldown-free

29.4 Transition

$hold[i] = \max(hold[i - 1], rest[i - 1] - price[i])$

$sold[i] = hold[i - 1] + price[i]$

$rest[i] = \max(rest[i - 1], sold[i - 1])$

29.5 C++ Code

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int hold = -prices[0];
        int sold = 0;
        int rest = 0;

        for (int i = 1; i < prices.size(); i++) {
            int prevHold = hold;
            int prevSold = sold;
            int prevRest = rest;

            hold = max(prevHold, prevRest - prices[i]);
            sold = prevHold + prices[i];
            rest = max(prevRest, prevSold);
        }

        return max(sold, rest);
    }
};
```

29.6 Lesson

Some DP problems require multiple states per index.
This becomes common in stock DP.

30. Full Problem Walkthrough 13 — Edit Distance, Preview

30.1 Problem Summary

Given two words, return minimum operations to convert word1 to word2.

Allowed:

- insert
- delete
- replace

30.2 State

$dp[i][j]$ = minimum operations to convert first i chars of word1 into first j chars of word2

30.3 Transition

If characters match:

$dp[i][j] = dp[i - 1][j - 1]$

Otherwise:

$dp[i][j] = 1 + \min(\begin{array}{l} dp[i - 1][j], \text{ delete} \\ dp[i][j - 1], \text{ insert} \\ dp[i - 1][j - 1] \text{ replace} \end{array})$

30.4 C++ Code

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        int m = word1.size();
        int n = word2.size();

        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 0; i <= m; i++) {
            dp[i][0] = i;
        }

        for (int j = 0; j <= n; j++) {
            dp[0][j] = j;
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1[i - 1] == word2[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
```

```

        dp[i][j] = 1 + min({
            dp[i - 1][j],
            dp[i][j - 1],
            dp[i - 1][j - 1]
        });
    }
}

return dp[m][n];
}
};

```

30.5 Lesson

String DP often uses:

$dp[i][j]$ = answer for prefixes

31. Full Problem Walkthrough 14 — Partition Equal Subset Sum, Preview

31.1 Problem Summary

Given nums, return true if it can be partitioned into two subsets with equal sum.

31.2 DP Transformation

If total sum is odd, impossible.

Otherwise need subset with sum:

target = total / 2

This becomes 0/1 knapsack possibility DP.

31.3 State

$dp[s]$ = true if some subset can make sum s

31.4 C++ Code

```

class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int total = accumulate(nums.begin(), nums.end(), 0);

        if (total % 2 == 1) {
            return false;
        }
    }
}

```

```

int target = total / 2;

vector<bool> dp(target + 1, false);
dp[0] = true;

for (int x : nums) {
    for (int s = target; s >= x; s--) {
        dp[s] = dp[s] || dp[s - x];
    }
}

return dp[target];
}
};

```

31.5 Lesson

Knapsack DP often loops backwards when each item can be used once.

32. Full Problem Walkthrough 15 — Burst Balloons, Preview

32.1 Problem Summary

Given balloons with numbers.

When you burst balloon i , gain:

$$\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$$

Return maximum coins.

32.2 Why This Is DP

The order of bursting matters.

Instead of choosing first balloon, choose the **last** balloon in an interval.

This becomes interval DP.

32.3 State

$$\text{dp}[l][r] = \text{max coins from bursting balloons between } l \text{ and } r$$

32.4 Transition

Choose last balloon k between l and r :

$$\text{dp}[l][r] = \max(\text{dp}[l][r], \text{dp}[l][k] + \text{nums}[l] * \text{nums}[k] * \text{nums}[r] + \text{dp}[k][r])$$

32.5 Lesson

Some DP problems become easier if you reverse perspective:

choose last instead of first

This is common in interval DP.

33. Common DP Edge Cases

33.1 Empty Input

Decide if answer is:

0, false, true, or empty

33.2 One Element

Often base case.

33.3 Impossible State

Use large value:

const int INF = 1e9;

33.4 Integer Overflow

Use long long when counting ways or summing large values.

33.5 Negative Values

Some DP transitions need careful initialization.

Example:

Maximum Subarray should initialize to nums[0], not 0.

33.6 Modulo

Counting DP often needs:

MOD = 1e9 + 7

34. Common DP Mistakes

Mistake 1 — Undefined State

If you cannot clearly say what dp[i] means, your solution will likely fail.

Mistake 2 — Wrong Base Case

Incorrect base cases produce wrong answers even if transition is correct.

Mistake 3 — Wrong Loop Order

Knapsack can change from 0/1 to unbounded just by loop direction.

Mistake 4 — Using Greedy When DP Is Needed

Coin Change is a common example.

Mistake 5 — Not Handling Impossible States

For minimum problems, initialize with INF.

Mistake 6 — Forgetting Space Optimization Dependencies

Only optimize space when you know which previous states are needed.

35. DP Interview Explanation Templates

35.1 General DP

I define $dp[state]$ as the answer to a smaller subproblem. Then I derive how this state depends on previous states, initialize base cases, and fill the table in an order where dependencies are already computed.

35.2 Memoization

I start from recursion and cache each state. If the same state appears again, I return the cached result instead of recomputing it.

35.3 Tabulation

I build the answer bottom-up from base cases, filling smaller states before larger states.

35.4 Min/Max DP

For each state, I consider all valid previous choices and take min or max depending on the objective.

35.5 Boolean DP

$dp[state]$ represents whether a condition is possible. Transitions usually use OR over possible choices.

36. Complexity Patterns

Fibonacci-style	$O(n)$	$O(1)$ or $O(n)$
1D amount DP	$O(n * \text{target})$	$O(\text{target})$
Grid DP	$O(\text{rows} * \text{cols})$	$O(\text{cols})$ possible
String prefix DP	$O(m * n)$	$O(m * n)$
LIS basic DP	$O(n^2)$	$O(n)$
Knapsack DP	$O(n * \text{capacity})$	$O(\text{capacity})$
Interval DP	$O(n^3)$	$O(n^2)$
Bitmask DP	$O(n * 2^n)$ or worse	$O(2^n)$
Tree DP	$O(n)$ often	$O(\text{height})$
Digit DP	$O(\text{digits} * \text{states})$	memo table

37. Time-Boxed Practice Guidance

Intro DP

Target:

15–30 minutes

Problems:

- Fibonacci
- Climbing Stairs
- Min Cost Climbing Stairs
- House Robber

Medium DP Foundations

Target:

30–50 minutes

Problems:

- Coin Change
- Word Break
- Decode Ways
- Unique Paths
- Triangle

Hard DP Preview

Target:

50–90 minutes

Problems:

- Edit Distance
- Partition Equal Subset Sum
- Burst Balloons
- Stock With Cooldown

38. Expanded Practice Set

38.1 Must-Do Easy

23. Fibonacci Number
24. Climbing Stairs
25. Min Cost Climbing Stairs
26. N-th Tribonacci Number
27. Pascal's Triangle
28. Maximum Subarray

38.2 Must-Do Medium

29. House Robber
30. House Robber II
31. Unique Paths
32. Unique Paths II
33. Minimum Path Sum
34. Coin Change
35. Coin Change II
36. Decode Ways
37. Word Break
38. Longest Increasing Subsequence
39. Triangle
40. Partition Equal Subset Sum

38.3 Strong Interview Practice

41. Edit Distance

- 42. Longest Common Subsequence
- 43. Best Time to Buy and Sell Stock with Cooldown
- 44. Target Sum
- 45. Palindromic Substrings
- 46. Longest Palindromic Subsequence
- 47. Interleaving String
- 48. Perfect Squares

38.4 Hard / Stretch

- 49. Burst Balloons
 - 50. Minimum Cost to Cut a Stick
 - 51. Regular Expression Matching
 - 52. Wildcard Matching
 - 53. Distinct Subsequences
 - 54. Scramble String
 - 55. Stone Game III
 - 56. Count All Possible Routes
-

39. Suggested Study Order

Step 1 — Fibonacci-Style DP

Start with:

- 57. Fibonacci
- 58. Climbing Stairs
- 59. Min Cost Climbing Stairs
- 60. Tribonacci

Goal:

Learn recurrence and base cases.

Step 2 — Choose / Skip DP

Solve:

- 61. House Robber
- 62. Maximum Subarray
- 63. Best Time to Buy and Sell Stock with Cooldown

Goal:

Learn decision states.

Step 3 — Grid DP

Solve:

- 64. Unique Paths
- 65. Unique Paths II
- 66. Minimum Path Sum
- 67. Triangle

Goal:

Learn 2D movement transitions.

Step 4 — Amount / Target DP

Solve:

- 68. Coin Change
- 69. Coin Change II
- 70. Partition Equal Subset Sum
- 71. Target Sum

Goal:

Learn target-based states.

Step 5 — String Prefix DP

Solve:

- 72. Decode Ways
- 73. Word Break
- 74. LCS
- 75. Edit Distance

Goal:

Learn prefix states.

Step 6 — Advanced Preview

Solve:

- 76. Burst Balloons
- 77. Longest Palindromic Subsequence
- 78. Stock DP variants

Goal:

Prepare for specialized DP chapters.

40. Pattern Decision Table

ways to reach step	dp[i] ways	Climbing Stairs
min cost to reach step	dp[i] min cost	Min Cost Stairs
choose or skip adjacent	dp[i] best till i	House Robber
grid paths	dp[r][c]	Unique Paths
min path in grid	dp[r][c] min sum	Minimum Path Sum
amount target	dp[amount]	Coin Change
string segmentation	dp[i] prefix valid	Word Break
decoding string	dp[i] prefix ways	Decode Ways
subsequence ending at i	dp[i] ending here	LIS
two string prefixes	dp[i][j]	Edit Distance
subset sum possible	dp[sum] boolean	Partition Equal Subset
interval decision	dp[l][r]	Burst Balloons
multiple modes	state variables	Stock Cooldown

41. Mini Problem 1 — N-th Tribonacci Number

```
class Solution {
public:
    int tribonacci(int n) {
        if (n == 0) {
            return 0;
        }

        if (n <= 2) {
            return 1;
        }

        int a = 0;
        int b = 1;
        int c = 1;

        for (int i = 3; i <= n; i++) {
            int d = a + b + c;
            a = b;
            b = c;
            c = d;
        }

        return c;
    }
};
```

Pattern:

$$dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3]$$

42. Mini Problem 2 — Perfect Squares

```
class Solution {
public:
    int numSquares(int n) {
        vector<int> dp(n + 1, 1e9);
        dp[0] = 0;
```

```

    for (int i = 1; i <= n; i++) {
        for (int square = 1; square * square <= i; square++) {
            dp[i] = min(dp[i], dp[i - square * square] + 1);
        }
    }

    return dp[n];
}
};

```

Pattern:

minimum count to form amount

Similar to Coin Change.

43. Mini Problem 3 — Unique Paths Space Optimized

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n, 1);

        for (int r = 1; r < m; r++) {
            for (int c = 1; c < n; c++) {
                dp[c] += dp[c - 1];
            }
        }

        return dp[n - 1];
    }
};

```

Pattern:

dp[c] = from top + from left

44. Mini Problem 4 — House Robber II Preview

Circular houses.

Cannot rob first and last together.

Solve twice:

79. rob houses 0..n-2

80. rob houses 1..n-1

```
class Solution {
public:
    int robLinear(vector<int>& nums, int left, int right) {
        int prev2 = 0;
        int prev1 = 0;

        for (int i = left; i <= right; i++) {
            int curr = max(prev1, prev2 + nums[i]);
            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }

    int rob(vector<int>& nums) {
        int n = nums.size();

        if (n == 1) {
            return nums[0];
        }

        return max(robLinear(nums, 0, n - 2),
                   robLinear(nums, 1, n - 1));
    }
};
```

45. Common Edge Case Checklist

Before submitting DP code, test:

```
n = 0
n = 1
empty array
single element
all zeros
all negatives
target = 0
impossible target
large target
```

odd total sum
string starts with 0
string has invalid 0
one-row grid
one-column grid

46. How to Debug DP

If DP fails:

Step 1

Print your state meaning.

What does `dp[i]` mean?

Step 2

Check base cases.

Step 3

Manually fill small example.

Step 4

Check transition dependencies.

Step 5

Check loop order.

Step 6

Check impossible-state initialization.

Step 7

Check final answer location.

Common final answer locations:

`dp[n]`
`dp[n - 1]`
`max(dp)`
`dp[m][n]`
`dp[target]`

47. What to Master Before Chapter 35

Before moving to 1D Dynamic Programming, make sure you can:

- Explain overlapping subproblems
- Explain optimal substructure
- Define DP state clearly
- Write recurrence relation
- Identify base cases
- Convert recursion to memoization
- Convert memoization to tabulation

- Space optimize Fibonacci-style DP
 - Solve Climbing Stairs
 - Solve Min Cost Climbing Stairs
 - Solve House Robber
 - Solve Coin Change
 - Solve Unique Paths
 - Solve Word Break
 - Solve Decode Ways
 - Know when greedy fails and DP is needed
 - Explain DP complexity
-

48. Final Cheat Sheet

DP Framework

State:

dp[...] = ?

Transition:

dp[...] = function of smaller states

Base:

known initial values

Answer:

where final answer is stored

Order:

which loops compute dependencies first

Memoization Template

```
int solve(int state) {
    if (base case) {
        return baseValue;
    }

    if (memo[state] != unknown) {
        return memo[state];
    }

    int answer = 0;

    for (choice : choices) {
        answer = combine(answer, solve(nextState));
    }
}
```

```
memo[state] = answer;
return answer;
}
```

Tabulation Template

```
vector<int> dp(n + 1);

dp[base] = baseValue;

for (int state = first; state <= last; state++) {
    dp[state] = transition from previous states;
}

return dp[answerState];
```

Min DP Initialization

```
const int INF = 1e9;
vector<int> dp(n + 1, INF);
```

Boolean DP Initialization

```
vector<bool> dp(n + 1, false);
dp[0] = true;
```

Count DP Initialization

```
vector<int> dp(n + 1, 0);
dp[0] = 1;
```

49. Core Takeaway

Dynamic Programming is about solving repeated subproblems once.

The most important skill is defining the state.

Always start by saying:

dp[state] means...

Then derive:

How can this state be computed from smaller states?

DP problems usually ask for:

- minimum
- maximum
- count
- true/false
- longest
- shortest

The DP process is:

define state
write recurrence
set base cases
choose computation order
return answer
optimize space if possible

If you master this foundation, the specialized DP chapters become much easier.